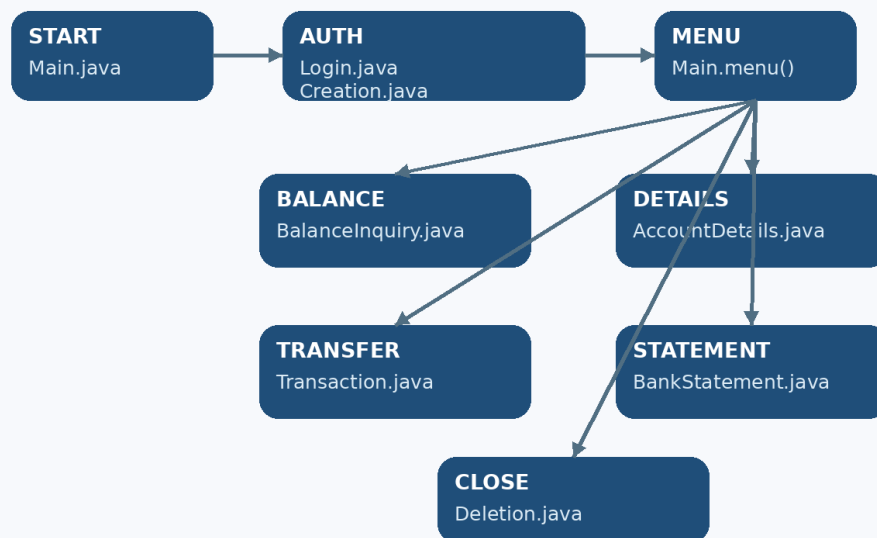


PROJECT REPORT

Bank Management System by Java

A Console-Based Banking Application Using Object-Oriented Java

System Flow and Module Navigation



Prepared as a structured academic analysis of the public GitHub implementation.

Project source: <https://github.com/Vedantt-Patel/Bank-Management-System-OOP-JAVA>

1. Executive Summary

Bank Management System by Java is a command-line banking application designed to demonstrate how core banking operations can be modelled with Java classes, interfaces, file handling, input validation, and menu-driven control flow. The implementation supports account creation, login, balance inquiry, personal account details, fund transfers, transaction history, logout, and account closure.

What the implementation is best viewed as

A learning-oriented banking simulation: it demonstrates OOP structure and file-based persistence rather than production-grade banking security or distributed financial infrastructure.

2. Problem Statement

Students learning object-oriented programming often understand individual concepts in isolation but struggle to connect them into a complete application. A banking system is a useful case because it naturally combines user identity, state, validation, transactions, persistence, and navigation. This project packages those ideas into a small interactive console system.

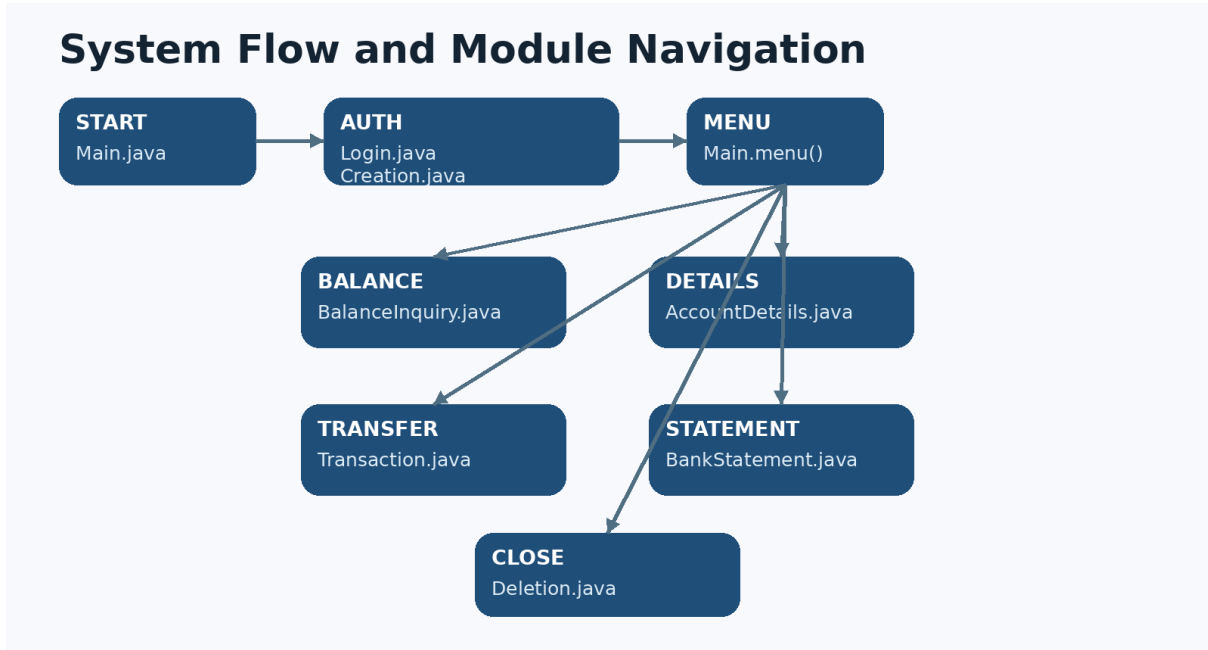
3. Objectives

- Model common banking actions through separate Java classes and methods.
- Apply interfaces to define login and account-creation contracts.
- Persist user credentials, customer details, balances, and transaction records using text files.
- Provide a simple menu-driven user experience that can be executed from a Java environment.
- Create a project that is small enough to study while still covering multiple OOP and I/O concepts.

4. Scope of the System

Area	Included	Current Boundary
Identity	Account number + password login	Credentials are stored in a plain text file
Account lifecycle	Create, view, close	No role hierarchy or approval workflow
Money movement	Account-to-account transfer	No external payment rails or bank integration
Statements	Per-account text-based transaction history	Basic console display
Persistence	Text files under db/	No relational or cloud database
Interface	Console / Scanner input	No GUI or web client

5. System Architecture



The runtime begins in Main.main(), displays an entry menu, and routes the user to login or account creation. After authentication, Main.menu(accNo) delegates each selected operation to its corresponding class.

Primary Classes and Responsibilities

Class	Responsibility
Main	Entry point, intro screen, menu routing, and delegation to feature classes.
Login	Reads credentials and validates account number/password combinations.
Creation	Collects customer information, validates phone/email, creates account number, and writes initial records.
BalanceInquiry	Finds and displays the current balance for an account.
AccountDetails	Reads customer data and displays the stored account profile.
Transaction	Validates receiver and sender balance, updates balances, and records debit/credit statements.
BankStatement	Reads a per-account statement file and prints formatted transaction history.
Deletion	Confirms account closure and removes the account from core text databases.

Program Entry and Navigation

Entry point: Main.java

```

public static void main(String[] args) throws IOException {
    intro();
}

```

Main.java acts as the controller for the application flow. The menu uses a switch statement to invoke the appropriate feature class, then most feature methods route the user back to Main.menu(accNo).

6. Functional Modules

Account creation

Creation.java collects name, DOB, gender, address, phone, email, identification number, and password; it then writes credentials, balance, and profile information to files.

Authentication

Login.java scans db/credentials.txt and compares the entered account number and password with stored records.

Balance inquiry

BalanceInquiry.java scans db/balanceDB.txt and displays the matching balance.

Account details

AccountDetails.java reads db/userDB.txt and prints the matching profile fields.

Fund transfer

Transaction.java checks recipient existence and sender balance, changes both balances, then writes debit and credit entries with date/time.

Bank statement

BankStatement.java reads db/Bank Statement/acc_<account>.txt and renders a tabular statement in the console.

Account closure

Deletion.java asks for confirmation and removes matching records from credentials, user, and balance files.

7. Object-Oriented Programming Analysis

The code demonstrates OOP at a practical introductory level. The strongest examples are abstraction through interfaces, encapsulation of functionality inside classes, and composition through Main delegating work to specialised objects. Classical inheritance is not the dominant design pattern in this implementation.

Concept	Where it appears	How it contributes
Classes / Objects	Main, Login, Creation, Transaction, etc.	Encapsulate related data-processing and behaviour by feature.
Abstraction	Authentication and AccountCreation interfaces	Expose method contracts without tying callers to implementation details.
Encapsulation	Methods such as balanceInquiryFun(), transactionFun()	Keeps feature logic inside the class responsible for it.
Modularity	One class per major banking operation	Makes the code easier to inspect, test, and later refactor.
Exception handling	throws IOException and try/catch blocks	Acknowledges file operations as failure-prone.
Polymorphism / inheritance	Limited in current code	Not a major mechanism in this version; future versions could model Account types.

Interfaces Used in the Project

Representative interface structure

```
interface Authentication {
    void loginAuth(int accNo, String pass) throws IOException;
}

interface AccountCreation {
    void createAccFun() throws IOException;
    ...
}
```

Creation implements AccountCreation, while Login provides the authentication behaviour defined by Authentication. This gives the project a clear introductory example of programming against a contract.

8. Data Storage Design

Instead of a database engine, the project maintains a small set of text files. This keeps setup simple and makes every record visible to a learner, but it also means that consistency, concurrency, security, and schema management are largely handled by application code.

File / Path	Role	Typical Record
db/credentials.txt	Authentication data	account number + password
db/userDB.txt	Customer profile data	account number + personal fields
db/balanceDB.txt	Current balances	account number + balance
db/Bank Statement/acc_<id>.txt	Transaction history	description, type, amount, remarks, date, time

Important design observation

The same logical account is represented across multiple files. Account closure therefore has to remove matching records from several data stores to keep the system coherent.

9. Key Workflows

Account creation workflow

- Generate the next account number from the last credentials record.
- Collect customer information from the console.
- Validate phone number format and email syntax.
- Append credentials, an initial balance, and profile information to their respective files.
- Return the user to the main banking menu.

Fund transfer workflow

- Read receiver account number, amount, and remarks.
- Verify that the receiver exists.
- Verify that the sender has sufficient balance.
- Rewrite the balance file with updated sender and receiver balances.
- Append debit and credit records to the two statement files.

Account closure workflow

- Ask the user for confirmation.
- Remove the account from the credentials file.
- Remove the profile and balance records from their files.
- Exit the application after successful deletion in the current flow.

10. Validation and Error Handling

Scenario	Current Behaviour	Observation
Invalid main-menu choice	Redisplays intro/menu	Simple recovery through recursion
Unknown account at login	Shows account-does-not-exist message and retries	Credential scan is file-based
Wrong password	Shows incorrect-password message and retries	No lockout/rate limit
Invalid phone number	Rejects unless exactly 10 numeric characters	Basic input validation
Invalid email	Checks a lightweight regex pattern	Not a full RFC-grade validator
Receiver missing	Blocks transfer	Prevents transfer to an unrecognised account
Insufficient balance	Blocks transfer	Core financial rule is enforced
Missing statement	Shows no-transaction message	Handled with FileNotFoundException

Strengths of the Current Version

- Feature separation is easy for beginners to understand.
- Interfaces introduce abstraction without making the application overly complex.
- The transfer workflow checks receiver validity and available funds before changing balances.
- Transaction history captures both sides of a transfer and includes date/time information.
- Text persistence makes the project easy to inspect without installing a database.

Current Limitations and Risks

- Credentials are stored in plaintext, so this should not be treated as secure authentication.
- Customer identification and personal data are also stored as plain text.
- The project has no transactional rollback mechanism if one file write succeeds and another fails.
- The balance and profile data are distributed across separate files, creating consistency risks.
- Input handling is mostly interactive and can be vulnerable to malformed data or unexpected numeric input.
- The initial balance is generated randomly rather than based on a deposit workflow.
- There is no explicit role-based access model, audit log, database indexing, or concurrency control.

11. Testing Strategy

A practical evaluation can be built around scenario-based functional tests. Each test starts from a known set of text files and verifies both console behaviour and resulting file state.

Test ID	Scenario	Expected Result
T01	Create valid account	Account number generated; credentials, profile, and balance records written
T02	Login with correct credentials	Login succeeds and account menu opens
T03	Login with incorrect password	Access denied and login retried
T04	Check balance	Correct balance for logged-in account

		displayed
T05	Transfer valid amount	Sender decreases, receiver increases, both statements updated
T06	Transfer above balance	Transfer rejected; balances unchanged
T07	Transfer to invalid account	Transfer rejected
T08	View statement with no history	Friendly no-transaction path
T09	Close account	Core records removed after confirmation

12. Future Enhancement Roadmap

Phase	Upgrade	Benefit
1	Refactor into src/main/java and data directories	Cleaner repository structure and buildability
2	Introduce Account, Customer, Transaction domain classes	Stronger object modelling and testability
3	Replace text files with SQLite / PostgreSQL	Integrity, querying, scalability
4	Hash passwords and protect personal data	Security baseline
5	Add JUnit tests and input-validation utilities	Regression safety
6	Add GUI or web/API layer	Improved usability and extensibility

13. Conclusion

Bank Management System by Java is a compact but meaningful OOP project because it connects classes, interfaces, file handling, validation, navigation, and state updates into a single working scenario. Its greatest value is educational: every major operation is visible in a small codebase, making it a useful foundation for the more structured clone and refactor planned next.

Source note

This report analyses the implementation visible in the referenced GitHub repository and intentionally distinguishes current behaviour from recommended future improvements.

14. References

<https://github.com/Vedantt-Patel/Bank-Management-System-OOP-JAVA>

Java platform concepts referenced in the implementation: java.io, java.util.Scanner, java.time.LocalDate / LocalDateTime, DateTimeFormatter.